Name: Prathamesh Arvind Jadhav
Roll No: 3014

EXPERIMENT NO-5

#traffic_data_exploration.py

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import cv2
import numpy as np
from sklearn.model_selection import train_test_split

data_dir = "D:\Deep Learning/05-Experiment-5/traffic/traffic_Data/DATA"
classes = os.listdir(data_dir)

data = []
labels = []

for i, folder in enumerate(classes):
    folder_path = os.path.join(data_dir, folder)
    for img_name in os.listdir(folder_path):
        img_path = os.path.join(folder_path, img_name)
        try:
            image = cv2.imread(img_path)
            image = cv2.resize(image, (30, 30))
```

```
            data.append(image)
            labels.append(int(folder))
        except:
            print(f"Error loading image: {img_path}")

X = np.array(data)
y = np.array(labels)

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

np.save("X_train.npy", X_train)
np.save("X_val.npy", X_val)
np.save("y_train.npy", y_train)
np.save("y_val.npy", y_val)

print("Data preprocessing completed. Shapes:")
print("X_train:", X_train.shape, "y_train:", y_train.shape)
print("X_val:", X_val.shape, "y_val:", y_val.shape)

#cnn_model.py
import numpy as np
import tensorflow as tf
```

Name: Prathamesh Arvind Jadhav
Roll No: 3014

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout

from tensorflow.keras.utils import to_categorical


# Load the data

X_train = np.load("X_train.npy")

y_train = np.load("y_train.npy")

X_val = np.load("X_val.npy")

y_val = np.load("y_val.npy")


# Filter out invalid labels (labels >= 43)

train_filter = y_train < 43

X_train = X_train[train_filter]

y_train = y_train[train_filter]


val_filter = y_val < 43

X_val = X_val[val_filter]

y_val = y_val[val_filter]


# One-hot encode the labels

y_train = to_categorical(y_train, num_classes=43)

y_val = to_categorical(y_val, num_classes=43)
```

Name: Prathamesh Arvind Jadhav
Roll No: 3014

```
# Build the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(30, 30, 3)),
    MaxPooling2D(2, 2),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(2, 2),
    Dropout(0.25),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(43, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=15, validation_data=(X_val, y_val),
batch_size=64)

# Save the model
```

```
model.save("traffic_model.h5")

print("Model saved as traffic_model.h5")


#app.py

import streamlit as st

import numpy as np

from tensorflow.keras.models import load_model

from PIL import Image


# Load the trained model

model = load_model("traffic_model.h5")


# Class labels

classes = {

    0: 'Speed limit (20km/h)', 1: 'Speed limit (30km/h)', 2: 'Speed limit (50km/h)',

    3: 'Speed limit (60km/h)', 4: 'Speed limit (70km/h)', 5: 'Speed limit (80km/h)',

    6: 'End of speed limit (80km/h)', 7: 'Speed limit (100km/h)', 8: 'Speed limit
(120km/h)',

    9: 'No passing', 10: 'No passing for vehicles over 3.5 metric tons',

    11: 'Right-of-way at the next intersection', 12: 'Priority road',

    13: 'Yield', 14: 'Stop', 15: 'No vehicles', 16: 'Vehicles over 3.5 metric tons
prohibited',

    17: 'No entry', 18: 'General caution', 19: 'Dangerous curve to the left',
```

```
    20: 'Dangerous curve to the right', 21: 'Double curve', 22: 'Bumpy road',
    23: 'Slippery road', 24: 'Road narrows on the right', 25: 'Road work',
    26: 'Traffic signals', 27: 'Pedestrians', 28: 'Children crossing',
    29: 'Bicycles crossing', 30: 'Beware of ice/snow', 31: 'Wild animals crossing',
    32: 'End of all speed and passing limits', 33: 'Turn right ahead',
    34: 'Turn left ahead', 35: 'Ahead only', 36: 'Go straight or right',
    37: 'Go straight or left', 38: 'Keep right', 39: 'Keep left',
    40: 'Roundabout mandatory', 41: 'End of no passing',
    42: 'End of no passing by vehicles over 3.5 metric tons'
}

# Streamlit UI
st.title("□ Traffic Sign Recognition")

uploaded_file = st.file_uploader("Upload a traffic sign image...", type=["jpg",
"png", "jpeg"])

if uploaded_file:
    img = Image.open(uploaded_file)
    st.image(img, caption="Uploaded Image", use_container_width=True)

    # Resize and preprocess
    img = img.resize((30, 30))
```

```
    img_array = np.array(img)


    # Ensure it's RGB format
    if img_array.shape == (30, 30, 3):
        img_array = np.expand_dims(img_array, axis=0)
        prediction = model.predict(img_array)
        predicted_class = np.argmax(prediction)
        st.success(f"Predicted Sign: {classes[predicted_class]}")
    else:
        st.error("Image format is incorrect. Please upload a valid RGB image.")
```

Output: